Technical report - "Autonomous Robots"

Autonomous Navigation and Target Reaching with the RosBot in Webots

Dhanvi Naga Anantha Kumar Nagandla
Meetkumar Shaileshbhai Vora
Rutvikkumar Babubhai Balar
Anil Paul Gunti[1]

[1]*OTH Amberg-Weiden, Study programme "Master of Artificial Intelligence for Industrial Applications"*

July 6, 2026

Abstract

We implemented an autonomous controller for the RosBot in the Webots maze environments. The robot starts without a map, builds an online occupancy grid, estimates its pose with wheel encoders, IMU yaw, and scan matching, and uses LiDAR plus RGB-D camera data to detect targets and hazards. The controller combines lightweight SLAM-style mapping, semantic layers for green floor, pillars, low bumps, and floating walls, Dijkstra planning on an inflated cost map, frontier exploration, pure-pursuit path following, and recovery behaviours. In our recorded tests, it completed all five mazes by reaching the blue pillar before the yellow pillar, without using Webots Supervisor information or changing the robot and world files.

Keywords: *autonomous robots, Webots, RosBot, SLAM, occupancy grid, frontier exploration, Dijkstra planning*

1. Introduction

The goal of this project was to implement autonomous navigation for the RosBot in Webots. After the simulation starts, the robot must first find and reach the blue pillar. Only after reaching the blue pillar is it allowed to go to the yellow pillar. The total mission time stops when the yellow pillar is reached. The main difficulty is that the robot has no full map at the beginning. While moving, it has to perceive walls, green ground, floating walls, narrow passages, too-narrow passages, and the two coloured pillars. The Webots Supervisor API is not allowed, so no ground-truth object positions or map data can be read directly. The controller therefore relies only on onboard sensors for mapping, localization, exploration, target detection, path planning, path following, and recovery. Our implementation is split into two controller files in each maze folder. The file `script.py` is the sensor and mapping layer. It initializes the Webots devices, estimates the robot pose, builds the map, detects special objects, and displays the debug map. The file `navigator.py` is the decision layer. It builds a cost map, chooses exploration targets, plans paths, executes the blue-to-yellow mission state machine, drives the motors, and performs recovery when the robot gets stuck.

2. Assignment Requirements and Constraints

2.1. Mission Requirements

The mission is to solve each Webots maze autonomously. The mission sequence is strict: after the simulation starts, the robot must first reach the blue pillar and only then navigate to the yellow pillar. The elapsed simulation time is measured in two parts: start to blue pillar and blue pillar to yellow pillar.

2.2. Environment Challenges

The provided environments contain normal walls, narrow passages, passages that are too narrow for the robot, green ground, and floating walls. Green ground is forbidden and must be treated as an obstacle. Floating walls are especially important because they are not always represented reliably by a simple two-dimensional LiDAR endpoint map.

2.3. Implementation Constraints

The solution must not use the Webots Supervisor API, must not modify the robot, and must not modify the provided environments. Therefore, all navigation decisions are based on onboard sensors and the controller's own map.

3. Task Decomposition

We split the assignment into the technical tasks shown in Table 1. Each row links one part of the assignment to the method that was implemented in the controller. This makes the table useful as a quick roadmap before the later sections describe the algorithms in more detail. The first rows focus on perception and map building, the middle rows show how special maze elements are detected, and the last rows explain how exploration, mission order, path following, and recovery are handled. The final column is included to make clear why each method is needed during a real run, not only which method was used.

Table 1. Project tasks and implemented methods.

Project task	Main method	Effect in the controller
Build a map	LiDAR occupancy grid with ray tracing and log-odds updates	Walls are marked as occupied, visible free space is cleared, and unknown space remains available for exploration.
Localize the robot	Differential-drive odometry, IMU yaw, and scan-matching correction	Encoders estimate travelled distance, the IMU stabilizes heading, and scan matching reduces drift against the map.
Avoid green ground	HSV colour mask plus depth projection into a green-floor map layer	Green cells become blocked in the cost map and nearby cells receive extra penalty.
Detect pillars	RGB colour detection, LiDAR distance validation, object-width check, curvature check, and repeated confirmation	The robot stores confirmed blue/yellow pillar positions and rejects most coloured walls or noise.
Handle floating walls	RGB-D height filtering, free-space-underneath test, dot memory, line completion, and back padding	Obstacles missed by normal LiDAR endpoint mapping are converted into blocked map lines.
Explore unknown maze	Frontier extraction, connected-component labeling, scoring, blacklisting, and target-biased frontiers	The robot expands known free space and can bias exploration toward blue or yellow.
Reach blue then yellow	Mission state machine with explicit `STATE_EXPLORE`, `STATE_GO_BLUE`, `STATE_GO_YELLOW`, and `STATE_FINISHED` states	The required order is enforced by the controller logic.
Drive safely	Inflated cost map, Dijkstra planning, pure-pursuit tracking, direct final approach, virtual bumper, tilt guard, and active recovery	The robot prefers safe routes, replans around new obstacles, and recovers from contact or deadlocks.

4. Overall Architecture

The controller is organized as a step-by-step navigation pipeline. Sensor readings are converted into pose and map information first, then the navigator uses those map layers to choose a safe action. This organization made debugging easier because mapping problems and planning problems could be checked separately. It also allowed the same controller structure to be reused in all five mazes while only changing small tuning values where necessary.

4.1. Controller Files

The active controller in all five worlds is named `script`. Each maze uses the pair `Maze*/controllers/script/script.py` and `Maze*/controllers/script/navigator.py`. The same architecture is used in all mazes, with small parameter changes for difficult passages, floating-wall padding, speed, and pillar confirmation thresholds. The split between these files keeps the implementation easier to understand: `script.py` prepares the sensor data, pose estimate, and map layers, while `navigator.py` turns this information into goals, paths, and wheel commands.

4.2. Sensor and Software Stack

Table 2 summarizes the sensors and software modules used by the controller. The first column names the input or module, the second gives its main purpose, and the third explains its role in the navigation pipeline. This matters because the robot receives no hidden map or object positions from Webots. Wheel encoders and the IMU support pose estimation, LiDAR provides geometric mapping and safety checks, and the RGB-D cameras add semantic information about pillars, green floor, low bumps, and floating walls. The software

modules combine these inputs into grid layers, frontiers, and planning costs. All entries are onboard sensors or local processing modules; no Supervisor information is used.

Table 2. Sensors and software modules used by the controller.

Input or module	Used for	Role in the methodology
Wheel encoders	Odometry	Estimate travelled distance from left and right wheel motion.
IMU yaw	Orientation	Stabilize the robot heading and reduce heading drift compared with pure wheel odometry.
IMU pitch/roll	Safety	Detect tilt or nose-up contact with missed low obstacles.
LiDAR	Mapping and safety	Detect walls, clear free space, validate pillar distance, and check the safety envelope.
RGB camera	Semantic perception	Detect blue/yellow pillar colours and green floor regions.
Depth camera	3D projection	Project camera detections into the map and detect low or floating obstacles.
`numpy`, OpenCV, `scipy`	Computation	Store grid layers, process image masks, label frontiers, and compute distance transforms.
Pygame view	Debugging	Visualize map layers, robot pose, current path, target frontiers, and safety zones during testing.

4.3. Runtime Pipeline

The control loop runs once per Webots time step. The main pipeline is:

1. update the robot pose from wheel encoders and IMU yaw;
2. check whether the robot has tilted or contacted a missed obstacle;
3. update LiDAR occupancy mapping when the sensors are warmed up and the robot is not rotating too quickly;
4. detect blue/yellow pillars, green floor, low bumps, and floating obstacles with camera data;
5. connect floating-obstacle detections into wall-like line segments and add safety padding;
6. pass all map layers to the navigator;
7. build an inflated cost map;
8. choose a mission action: explore, go to blue, go to yellow, recover, or finish;
9. plan with Dijkstra and follow the path with pure pursuit;
10. draw the debug visualization.

4.4. Main Data Flow

The data flow is sensor-driven rather than map-driven. The robot never receives a full environment map. Instead, the controller builds the information needed for navigation during the run:

$$\text{sensors} \rightarrow \text{pose estimate} \rightarrow \text{map layers} \rightarrow \text{cost map}$$
$$\rightarrow \text{frontier or pillar goal} \rightarrow \text{planned path} \rightarrow \text{wheel velocities}$$

5. Mapping and Localization Methodology

5.1. Lightweight SLAM-Style Approach

The mapping task was implemented with a lightweight SLAM-style pipeline based on occupancy-grid mapping [2]. It is not a full graph-SLAM system with loop-closure optimization. Instead, it combines local odometry, IMU heading, scan matching, and an occupancy grid. This level of mapping matched the scale of the bounded Webots mazes and supported online planning without Supervisor information.

5.2. Pose Estimation

5.2.1. Wheel Encoder Odometry

The RosBot has four wheel encoders. The controller averages the front and rear wheels on each side to estimate the left and right wheel travel. From these values it computes the forward displacement of the differential-drive base:

$$\Delta s = \frac{\Delta s_L + \Delta s_R}{2}. \tag{1}$$

This displacement is projected into the world frame using the current heading. The estimated position is stored as `x` and `y` in metres.

5.2.2. IMU Heading

Wheel odometry alone accumulates heading drift. Therefore, the yaw angle from the IMU is used as the robot heading `theta`. The controller uses the average between the previous heading and the new IMU yaw during the position update, which gives smoother motion integration while preserving the IMU heading as the main orientation estimate.

5.2.3. Scan Matching Correction

Small odometry errors are reduced with scan matching. Every few mapping steps, the controller tests small candidate pose offsets in `x`, `y`, and `theta`. For each candidate, LiDAR endpoints are projected into the current occupancy map and scored by how well they agree with already known wall cells. A correction is applied only if enough known cells are visible and the score improvement exceeds a threshold. This helped keep the map consistent without relying on global ground truth.

5.3. Occupancy Grid Mapping

5.3.1. Grid Definition

The map is a fixed `600 x 600` grid with a resolution of `0.02 m` per cell. The map is centered around the starting coordinate system. Each cell stores an occupancy value. Positive values mean obstacle evidence, negative values mean free-space evidence, and values around `0.5` are treated as unknown.

Table 3 lists the most important map layers. Keeping these layers separate made the system easier to debug and allowed the navigator to combine geometric and semantic hazards in one cost map.

Table 3. Main map layers used by the controller.

Layer	Meaning and use
`grid`	Main occupancy grid. Positive values represent obstacle evidence, negative values represent free-space evidence, and unknown cells remain near the initial value.
`locked_walls`	Cells with strong wall evidence that should not be cleared later by noisy free-space rays.
`lidar_only_walls`	Walls confirmed by LiDAR; used to prevent ray clearing through already confirmed obstacles.
`green_floor`	Forbidden floor cells detected from RGB-D camera data; treated as blocked by the navigator.
`floating_obstacles` and `floating_obstacle_memory`	Current and remembered floating-obstacle evidence. These cells are added to the blocked planning layer.
`floating_detection_dots` and `floating_back_padding`	Temporary evidence dots and extra safety padding used when completing floating walls into line obstacles.
`low_bump_votes`	Confirmation memory for low obstacles so that isolated depth noise does not become a permanent wall.
`mission_pillars`	Confirmed blue and yellow pillar positions used by the mission state machine.

5.3.2. LiDAR Ray Tracing

LiDAR is the main sensor for normal walls and free space. For each valid LiDAR beam, the endpoint is projected into the grid and marked as occupied. The cells between the robot and the endpoint are found with Bresenham ray traversal and marked as free. This creates both walls and corridors in the occupancy grid. Occupancy values are clipped between configured minimum and maximum limits so repeated observations remain numerically stable.

5.3.3. Locked Walls and Map Stability

Once a cell has enough wall evidence, it is stored in `locked_walls` and `lidar_only_walls`. Locked cells are not cleared by later free-space rays. This reduces cases where real walls disappear because of sensor noise or grazing LiDAR rays. After ray updates, tiny unknown gaps inside known free regions are filled, which made frontier detection and path planning more stable in testing.

6. Semantic Perception Methodology

6.1. Blue and Yellow Pillar Detection

6.1.1. Colour Segmentation

The RGB camera is used to detect the two mission targets. Sampled pixels are converted from RGB to HSV. Blue and yellow pixels are counted using tuned HSV ranges. The controller only continues if one colour has enough pixels and dominates the other colour by a threshold.

6.1.2. Distance and Shape Validation

Colour alone was not enough because coloured wall regions or reflections could be mistaken for pillars. For this reason, the pillar detector validates the colour blob with LiDAR. The lowest coloured pixel gives an approximate camera-based distance. LiDAR points in the same angular cone provide an independent distance estimate. If the two distances disagree too strongly, the detection is rejected.

The detector also estimates the object width from neighbouring LiDAR points. Objects that are too wide, too narrow, or too flat are rejected. A curvature or "bulge" check helps distinguish a round pillar from a flat coloured wall. After passing these checks, the pillar centre is estimated by adding the known pillar radius to the measured front-surface distance.

6.1.3. Confirmation and Map Integration

A pillar must be detected multiple times before it is accepted as a mission target. Confirmed detections are averaged to reduce noise. The final pillar position is stored in `mission_pillars`. The pillar itself is then marked as a small blocked circle in the occupancy grid, because the robot must drive near the pillar but not through it.

6.2. Green Ground Detection

Green ground is forbidden by the assignment, so it is represented as its own map layer. The controller examines the lower part of the RGB camera image, where floor pixels are visible. It applies an HSV green mask, cleans the mask with morphological opening and closing, and uses depth values to project green pixels into world coordinates. The corresponding grid cells are marked in `green_floor` and expanded slightly to fill small holes. In the navigator, green cells are hard-blocked, and cells near green ground receive additional cost.

6.3. Low Bump Detection

Some obstacles are low enough that they can be missed by simple LiDAR endpoint mapping but still block or destabilize the robot. The low-bump detector samples the depth camera, estimates point heights relative to the camera, filters for low obstacle heights, groups candidate cells with connected components, and uses a voting grid. Only cells confirmed across repeated frames are written into the occupancy map as locked obstacles. This prevents one noisy frame from creating a permanent false wall.

6.4. Floating Wall Detection

6.4.1. Problem

Floating walls are difficult because they may not create a normal wall shape in the LiDAR map. The robot must nevertheless treat them as obstacles if they are low enough to collide with the robot body.

6.4.2. RGB-D Height Filtering

The depth camera is sampled on a regular pixel grid. For each valid depth point, the controller estimates its height above the floor. Points are kept if they lie in the height range relevant for the robot and are closer than the maximum useful depth. The detector also checks a pixel below the candidate point. If the lower pixel is much farther away, there is likely free space below the object, which is evidence for a floating obstacle.

6.4.3. Memory, Line Completion, and Padding

Floating detections are stored as temporary dots with a time-to-live. Connected components of dots are analysed for linearity. If a cluster forms a sufficiently long line, principal direction estimation and Bresenham cells are used to complete it into a wall-like obstacle. The line can be snapped to horizontal or vertical directions when appropriate. Additional back padding is added behind accepted floating-wall lines so that the planner does not drive too close to them. Padding near confirmed pillars is cleared so pillars remain reachable.

7. Navigation Methodology

7.1. Cost Map Construction

The navigator converts the raw map layers into a planning cost map. Occupied cells, floating-obstacle memory, floating back padding, and green-floor cells are treated as blocked. The distance transform of the free space is used to inflate obstacles by the robot radius, so planned paths keep clearance from walls where the maze width allows it.

Valid free cells receive a cost based on clearance:

$$\text{cost} = 1 + \frac{w}{\text{clearance}^2}. \tag{2}$$

With this cost function, paths through the centre of corridors are cheaper than paths close to walls. Because some mazes contain narrow valid passages, the safety padding is tuned per maze to trade off clearance against the need to pass through valid narrow corridors.

7.2. Frontier-Based Exploration

7.2.1. Frontier Definition

When a pillar or route is not yet known, the robot explores with the frontier-based idea of moving toward boundaries between known and unknown space [3]. A frontier is a known free cell adjacent to unknown space. The navigator builds a frontier mask from the occupancy grid and labels connected frontier regions using connected-component labeling.

7.2.2. Frontier Selection

For each frontier region, the controller chooses a safe representative cell near the centre of the region. The utility score combines frontier size and distance from the robot. Large frontiers are useful because they can reveal more unknown space, while shorter paths are preferred for time efficiency. If the robot repeatedly chooses local frontiers without progress, a global escape mode prefers farther frontiers.

7.2.3. Blacklists and Target Bias

Frontiers that have been visited, failed, or repeatedly unreachable are blacklisted. Cooldowns prevent the robot from immediately retrying bad targets. If a pillar is known but no safe route to it exists, exploration is biased toward frontiers near that pillar. In practice, this made exploration more focused on opening a route to the current mission target.

7.3. Path Planning

Global paths are planned with Dijkstra search on the cost map [1]. The planner expands eight-connected neighbours. Straight moves have cost `1.0`, diagonal moves have cost `1.414`, and both are multiplied by the target cell cost. Infinite-cost cells are blocked.

The start and goal neighbourhoods are slightly relaxed so the robot is not trapped by its own inflated footprint or by the obstacle disk around a pillar. For pillar navigation, the robot does not plan to the pillar centre. Instead, it creates candidate approach cells around the pillar and plans to the closest safe candidate. This is necessary because the pillar itself is an obstacle.

7.4. Path Following

Planned paths are followed with a pure-pursuit-style controller [4]. The controller skips waypoints already reached, selects a lookahead point on the path, computes the heading error, and converts target linear and angular velocity into left and right wheel velocities. If the heading error is large, forward velocity is reduced to zero so the robot turns before driving forward. Motor speeds are clamped to the configured maximum.

7.5. Direct Final Approach

When the robot is close to a pillar, a direct approach is faster than following a longer grid path. However, direct driving is only allowed if the straight line to the pillar is clear in the cost map and in the hard blocked obstacle layer. If the direct path is blocked, the navigator routes around the obstacle with Dijkstra instead of blindly driving into the pillar area.

8. Mission State Machine

8.1. State Overview

The mission order is enforced by four states:

`STATE_EXPLORE` → `STATE_GO_BLUE` → `STATE_GO_YELLOW` → `STATE_FINISHED`

8.2. Explore State

The robot starts in `STATE_EXPLORE`. In this state it builds the map, searches for pillars, and selects frontiers. If only blue is known and yellow is still missing, the robot can start a rotational scan after repeated frontier failures. If both pillars are known but blue is not confidently reachable, exploration is biased toward route-opening frontiers near blue. The state changes to `STATE_GO_BLUE` only when blue is nearby or a route to blue is available.

8.3. Go to Blue State

In `STATE_GO_BLUE`, the robot must reach the blue pillar before doing anything else. It first tries to plan a routed path to a safe approach cell near blue. If no such route exists, it explores frontiers biased toward blue. When the robot is inside the handoff radius, it uses direct driving only if the final line is clear. The blue pillar is counted as reached only when the robot is within the blue reach threshold; being near a blocked or unknown approach does not count. After reaching blue, the state changes to `STATE_GO_YELLOW`.

8.4. Go to Yellow State

In `STATE_GO_YELLOW`, the robot repeats the same strategy for the yellow pillar. If a short reverse path away from the blue pillar was saved, it first uses that path to avoid starting the yellow route too close to blue. Then it plans a Dijkstra route to yellow, explores frontiers biased toward yellow if no route exists, and uses direct final approach only when the line is clear. When yellow is reached, the state becomes `STATE_FINISHED`.

8.5. Finished State

In `STATE_FINISHED`, all wheel motors are stopped. This is the mission-complete condition used for the timing results.

9. Safety and Recovery Methodology

9.1. Path Monitoring

The robot continuously checks the upcoming waypoints of the current path. If a waypoint ahead becomes infinitely expensive or too close to a newly detected obstacle, the path is cleared and replanned. This is important because floating obstacles, green ground, and low bumps can be discovered after a path was originally planned.

9.2. Hardware Safety Envelope and Virtual Bumper

A rectangular safety envelope is checked using LiDAR points in robot coordinates. If an obstacle appears inside the robot body envelope, the controller brakes and replans after repeated detections. A virtual bumper also checks highly occupied grid cells directly in front of the robot. This catches obstacles represented in the map even if the instantaneous LiDAR check does not trigger.

9.3. Stuck Detection and Active Recovery

While following a path, the navigator compares the robot position over time. If the robot barely moves for several checks while a path is active, it assumes wheel slip or physical blockage. Active recovery clears the current path, backs up for a short time, rotates, stops, and then requests new planning. The failed frontier or path can be blacklisted so the robot does not immediately repeat the same mistake.

10. Design Decisions and Trade-Offs

10.1. Why an Occupancy Grid Was Used

An occupancy grid was chosen because the maze is bounded, mostly planar, and well suited to LiDAR ray tracing. A full graph-SLAM system would add complexity that was not necessary for the assignment. The implemented approach still performs SLAM-style online mapping because localization and map building run together, but it stays lightweight enough for real-time Webots control.

10.2. Hard Obstacles Versus Soft Costs

The controller separates hard constraints from soft preferences. Walls, green floor, confirmed low bumps, and floating-wall memory are hard obstacles. Cells near obstacles remain traversable only if they are outside the inflated robot footprint, but they receive higher costs. As a result, the robot prefers wider corridors while still allowing narrow valid passages when necessary.

10.3. Exploration Versus Mission Driving

The robot does not immediately drive to every coloured object. First, a pillar must be confirmed. Then the navigator checks whether a safe path or clear final approach exists. If no route exists, the robot returns to exploration but biases frontiers toward the relevant pillar. This trade-off prevents both random exploration after a target is known and unsafe direct driving through unknown or blocked cells.

10.4. Recovery as Map Improvement

Recovery is not only a motor behaviour. When the robot tilts, slips, or contacts an unseen obstacle, the controller also updates the map by marking the probable problem area as blocked. Failed movements therefore become new map information and reduce repeated attempts through the same obstacle.

11. Maze-Specific Parameter Tuning

The same controller code and algorithm structure were used for all five mazes. Only parameter values were changed to match the layout of each maze. The changes were:

- **Maze 1:** used a higher maximum linear speed of `0.21 m/s`, a safety padding value of `-0.050 m`, stricter floating-wall anchoring, `30` floating-wall ray cells, `18` minimum floating-line cells, `10` back-padding cells, and a floating-obstacle minimum height of `0.08 m`.
- **Maze 2 and Maze 3:** used the conservative baseline settings: maximum linear speed `0.15 m/s`, safety padding `-0.040 m`, floating-obstacle minimum height `0.02 m`, `24` floating-wall ray cells, `13` minimum floating-line cells, `14` back-padding cells, and low-bump detection enabled.
- **Maze 4:** used the same baseline as Maze 2 and Maze 3, but with safety padding changed to `-0.070 m`. This made the planner more permissive in narrow valid passages.
- **Maze 5:** used maximum linear speed `0.15 m/s`, safety padding `-0.040 m`, only `2` required pillar detections instead of `3`, larger blue/yellow reached thresholds of `0.50 m`, and `12` floating-wall back-padding cells.

These differences are parameter tuning only. The mapping, perception, planning, mission logic, and recovery algorithms stayed the same.

12. Implementation Summary

Table 4 summarizes the implementation files and their roles.

Table 4. Implementation structure.

File or folder	Role in the solution
Sensor file: `script.py`	Webots device initialization, wheel/IMU odometry, LiDAR mapping, camera perception, green-floor detection, pillar detection, low-bump detection, floating-wall detection, map-layer maintenance, and Pygame visualization.
Navigator file: `navigator.py`	Cost-map generation, obstacle inflation, frontier exploration, Dijkstra path planning, pillar approach planning, mission state machine, pure-pursuit driving, direct final approach, stuck detection, and active recovery.

The implementation uses the Webots `controller` API for motors and sensors, `numpy` for grid maps, `pygame` for visualization, `colorsys` for HSV conversion, OpenCV for camera masks and connected components, `heapq` for Dijkstra's priority queue, and `scipy.ndimage` for distance transforms and frontier labeling.

13. Testing and Evaluation Methodology

13.1. Mission Success Criteria

A run was counted as solved only if the robot reached the blue pillar first, then reached the yellow pillar, and did not rely on prohibited Supervisor information. The controller's state machine enforces this order internally: the yellow mission state is only entered after the blue reach threshold has been satisfied.

13.2. Timing Measurement

Timing was recorded in the format required by the assignment: simulation start to blue pillar reached, and blue pillar reached to yellow pillar reached. The total time is the sum of these two intervals. The submitted videos should show the simulation time clearly so that the timings in Table 5 can be verified visually.

14. Results

Table 5 shows the recorded simulation timings. The table follows the required split between simulation start to blue pillar and blue pillar to yellow pillar.

Table 5. Recorded timing results.

Maze	Start–Blue	Blue–Yellow	Total	Status
1	2:23	0:55	3:18	Solved
2	0:38	1:07	1:45	Solved
3	1:40	0:46	2:26	Solved
4	1:53	1:15	3:08	Solved
5	0:41	0:41	1:22	Solved

15. Limitation

The controller works for the provided mazes in our tests, but it still has limitations. HSV thresholds depend on lighting and camera angle. Pillar detection can be delayed if a pillar is only partially visible. Floating-wall line completion depends on enough depth evidence and can be conservative in sparse views. Very narrow passages can still trigger contact and recovery. Finally, parameters are tuned per maze rather than learned automatically.

16. Academic Integrity

All project ideas, algorithm choices, testing, tuning, and final integration decisions were made by our group. OpenAI ChatGPT was used as a coding assistant for implementation, and the assisted work was reviewed and integrated by the group. No code from other student groups was copied, and the Webots Supervisor API was not used.

References

[1] E. W. Dijkstra, "A note on two problems in connexion with graphs", *Numerische Mathematik*, vol. 1, pp. 269–271, 1959. DOI: 10.1007/BF01386390

[2] A. Elfes, "Using occupancy grids for mobile robot perception and navigation", *Computer*, vol. 22, no. 6, pp. 46–57, 1989. DOI: 10.1109/2.30720

[3] B. Yamauchi, "A frontier-based approach for autonomous exploration", in *Proceedings of the IEEE International Symposium on Computational Intelligence in Robotics and Automation*, 1997, pp. 146–151. DOI: 10.1109/CIRA.1997.613851

[4] J. Morales, J. L. Martínez, M. A. Martínez, and A. Mandow, "Pure-pursuit reactive path tracking for nonholonomic mobile robots with a 2d laser scanner", *EURASIP Journal on Advances in Signal Processing*, vol. 2009, p. 935 237, 2009. DOI: 10.1155/2009/935237